

Dashboard Web – Cannoli Foodtech

Design de Software e Diagramas UML

Projeto Interdisciplinar — 4CCOMP — 4º Semestre 2026 — FECAP

Bruno Souza Lima — RA: 24026560

Felipe Toshio Yamaschita — RA: 24026779

Breno Costa do Nascimento — RA: 24026753

Vinícius Nishimura Reis — RA: 24026962

Disciplina: Engenharia de Software e Arquitetura de Sistemas

Professora Orientadora: Lucy Mary

Empresa Parceira: Cannoli Foodtech | Maio / 2026

1. Introdução

Este documento apresenta o design de software e os diagramas UML desenvolvidos para o Projeto Interdisciplinar do 4º semestre — o Dashboard Web da Cannoli Foodtech. O sistema reúne dados de pedidos, campanhas, clientes e lojas em painéis de indicadores, com acessos diferentes para o time interno da Cannoli e para os restaurantes parceiros.

O objetivo aqui é registrar as decisões de arquitetura que já foram tomadas ou estão em andamento (backend em FastAPI + frontend em React/Next.js), usando os diagramas UML para orientar a implementação e facilitar a manutenção futura.

2. Design de Software Aplicado

2.1 Padrão Arquitetural Adotado

O sistema usa arquitetura em três camadas, separando frontend, backend e banco de dados. Isso permite mexer em cada parte de forma independente conforme o projeto cresce:

Camada	Tecnologia	Responsabilidade
Apresentação (Frontend)	React/Next.js + TypeScript Recharts / ECharts	Renderização dos painéis, filtros, gráficos e interações do usuário
Aplicação (Backend)	FastAPI (Python) JWT + ORM	Autenticação, endpoints de KPIs, pipeline de dados, lógica de negócio
Dados (Banco)	PostgreSQL (SQLite no MVP)	Persistência de dados tratados, histórico, metas e logs de auditoria
Integração (ETL)	Pandas + SQL (polling / WebSocket)	Importação, limpeza, deduplicação e enriquecimento dos dados
Infra / DevOps	Docker + docker-compose GitHub Actions	Build, testes, deploy em staging e produção

2.2 Padrões de Design Utilizados

- **Repository Pattern** — Abstração do acesso ao banco de dados, isolando queries SQL da lógica de negócio nas camadas de serviço da API FastAPI.
- **Service Layer** — Lógica de cálculo de KPIs (ticket médio, taxa de conversão, RFM) encapsulada em serviços reutilizáveis, independente dos controllers HTTP.
- **JWT Middleware** — Autenticação e autorização por perfil (Admin vs Parceiro) implementadas via middleware na API, garantindo segurança real além do frontend.
- **Observer / Event-driven (Alertas)** — Módulo de alertas monitora KPIs e dispara notificações quando limiares configurados são atingidos (queda de recorrência, aumento de cancelamentos, variação de ticket médio).
- **ETL Pipeline** — Ingestão, limpeza e enriquecimento de dados em etapas sequenciais com validação de schema, deduplicação e log de linhas rejeitadas.

2.3 Principais Endpoints da API (Contrato)

A API é documentada via Swagger (acessível em /docs durante o desenvolvimento). Os principais endpoints já implementados ou em andamento:

Método	Rota	Descrição	Perfil
--------	------	-----------	--------

POST	/auth/login	Autenticação → JWT	Todos
POST	/auth/register	Cadastro de usuário	Admin
GET	/dashboard/kpis	KPIs agregados com filtros	Todos
GET	/campanhas	Listagem e métricas de campanhas	Todos
GET	/campanhas/{id}/ab	Resultado Teste A/B	Todos
GET	/clientes/segmentacao	Segmentação RFM e coortes	Admin
POST	/dados/importar	Upload CSV/Excel	Admin
GET	/financeiro/painel	Indicadores financeiros	Todos
GET	/alertas	Alertas ativos	Todos
PUT	/config/metast	Configuração de metas e limiares	Admin

3. Diagramas UML

3.1 Diagrama de Classes

O diagrama abaixo mostra as principais entidades do sistema, seus campos e os relacionamentos entre elas. Reflete o esquema do banco PostgreSQL e as classes do backend:

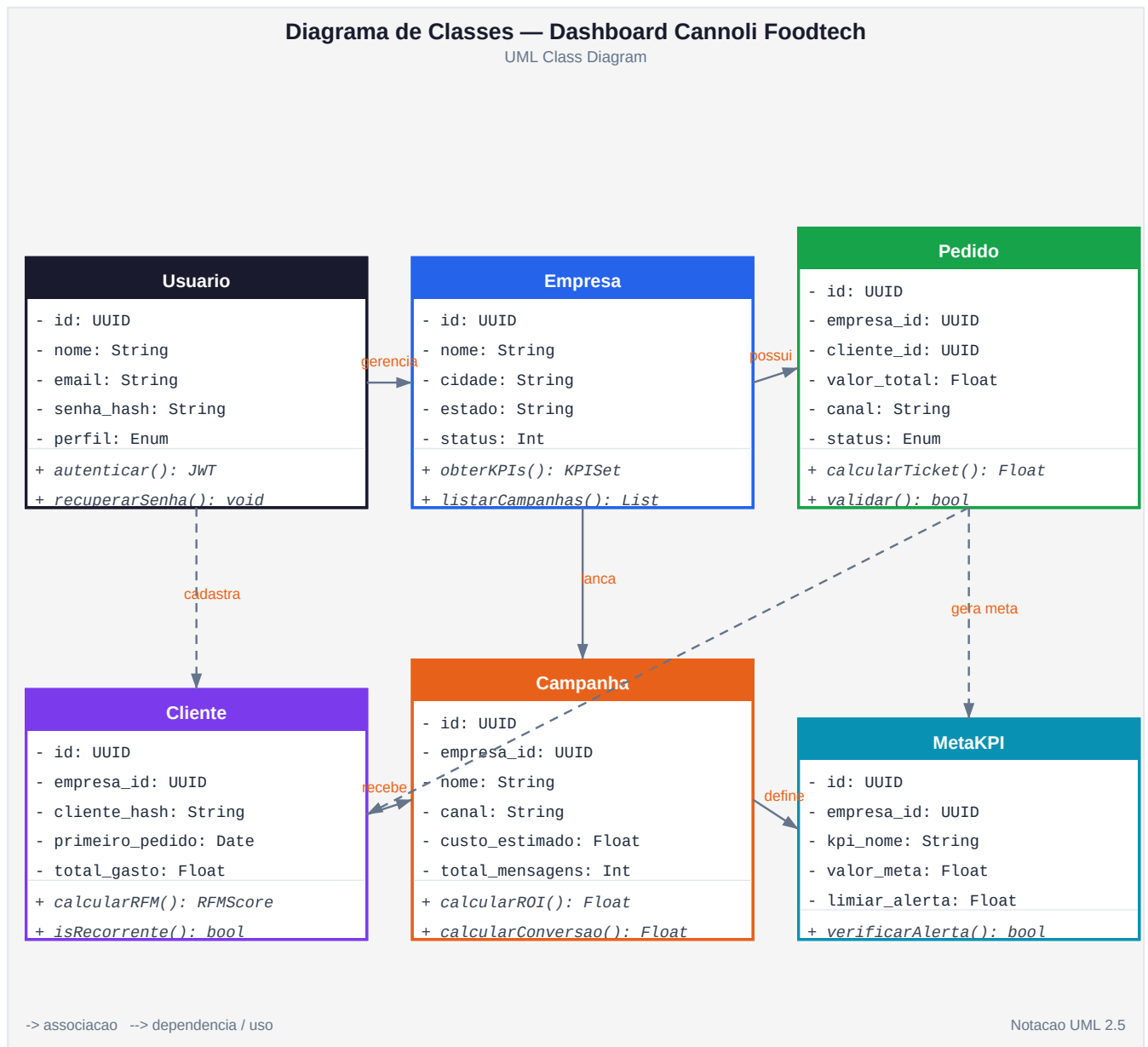


Figura 1 — Diagrama de Classes UML do Dashboard Cannoli Foodtech.

O diagrama conta com seis classes principais: **Usuario**, **Empresa**, **Pedido**, **Cliente**, **Campanha** e **MetaKPI**. A classe **Usuario** se associa a **Empresa** (que ela gerencia) e a **Cliente** (que ela cadastra). **Empresa** possui **Pedidos** e lança **Campanhas**, enquanto **Campanha** define metas em **MetaKPI** e é recebida por **Clientes**. **Pedido** referencia tanto a **Empresa** quanto ao **Cliente** que o realizou, e gera dados que alimentam as metas de KPI. Os métodos em itálico dentro de cada classe representam as operações que o backend expõe, como `calcularROI()`, `verificarAlerta()` e `autenticar()`, refletindo diretamente os endpoints da API.

3.2 Diagrama de Casos de Uso

O diagrama de casos de uso mostra quem usa o quê no sistema. Temos três atores: Admin Cannoli, Restaurante Parceiro e o Sistema ETL. As linhas tracejadas indicam dependência ou inclusão entre casos de uso:

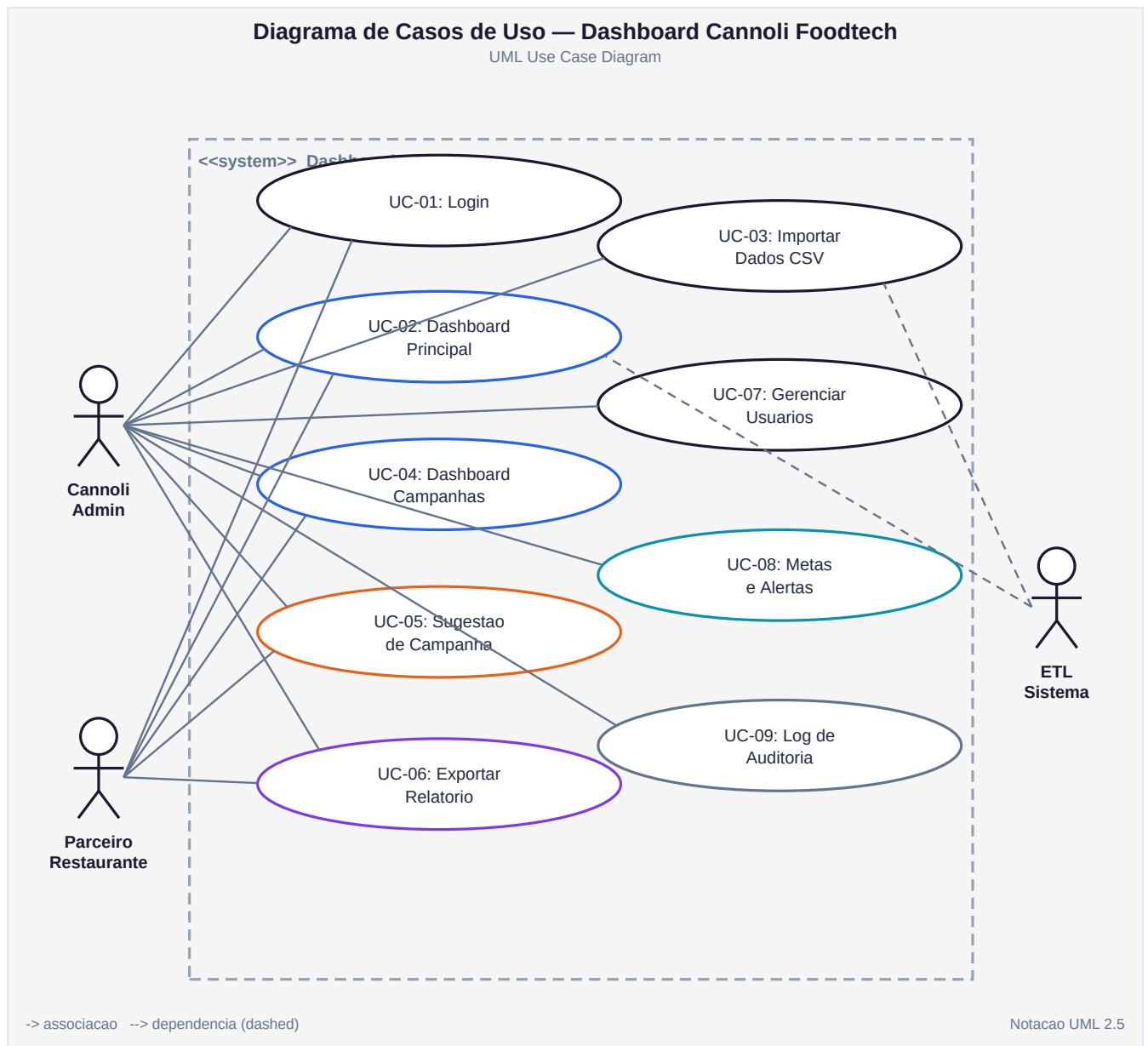


Figura 2 — Diagrama de Casos de Uso UML do Dashboard Cannoli Foodtech.

O Admin Cannoli tem acesso completo ao sistema, podendo fazer login, visualizar os dashboards principal e de campanhas, gerar sugestões de campanha, exportar relatórios, importar dados via CSV, gerenciar usuários, configurar metas e alertas e consultar o log de auditoria. O Restaurante Parceiro tem acesso restrito aos casos de uso operacionais — login, dashboards e exportação — sem acesso às funções administrativas. O Sistema ETL aparece como ator externo, responsável por acionar a importação de dados (UC-03) e atualizar o dashboard principal (UC-02) de forma automática via polling agendado, sem interação humana direta.

4. Entradas e Saídas do Sistema

4.1 Entradas (Inputs)

Entrada	Formato	Origem	Validações Aplicadas
Arquivos de pedidos (STOREORDER)	CSV	Plataforma Cannoli SaaS	Schema, tipos, duplicatas, totalamount > 0
Cadastro de clientes (CUSTOMER)	CSV	Plataforma Cannoli SaaS	gender='NI' se nulo, dates UTC normalizadas
Campanhas (CAMPAIGN)	CSV	Plataforma Cannoli SaaS	status válido, sendat ≥ createdat
Relação campanha-pedido (CAMPAIGNxORDER)	CSV	Plataforma Cannoli SaaS	status ∈ {2,3,4}, referências íntegras
Endereços de clientes (CUSTOMERADDRESS)	CSV	Plataforma Cannoli SaaS	UF com 2 chars, CEP numérico
Lojas (STORE)	CSV	Plataforma Cannoli SaaS	id único, status ativo
Credenciais de usuário	Form (JSON)	Interface Web	E-mail válido, senha ≥ 8 chars, hash bcrypt fator 10
Configuração de metas	Form (JSON)	Interface Admin	valor_meta > 0, limiar ∈ [0%, 100%]

4.2 Saídas (Outputs)

Saída	Formato	Destino	Observação
Cards de KPI (pedidos, receita, ticket, recorrência)	JSON → UI	Dashboard Web	Atualizado por polling; filtro por período/loja/canal
Gráfico de série temporal	JSON → Recharts	Dashboard Web	Sobreposição período anterior + variação %
Taxa de conversão de campanhas	JSON → UI	Dashboard Campanhas	status=4 / total enviado × 100
ROI simulado por campanha	JSON → UI	Dashboard Campanhas	(Receita – Custo) / Custo × 100
Segmentação RFM e coortes	JSON → UI	Painel de Clientes	Recência, Frequência, Valor monetário
Sugestão de campanha	JSON → UI	Módulo de Recomendação	Baseada em RFM e histórico de conversão
Alertas automáticos	Notificação → UI	Dashboard / E-mail	Queda recorrência, aumento cancelamentos
Relatório exportado	CSV / PDF	Download pelo usuário	Top produtos, campanhas, clientes drill-down
Log de auditoria	Tabela → UI	Admin Painel Administrativo	Login, importações, alterações de meta
Token JWT	JSON (response)	Frontend (localStorage)	Expira em 8h; configurável

5. KPIs Calculados — Resultados da Base Real

Os valores abaixo foram extraídos da base real da plataforma Cannoli (julho/2025 a janeiro/2026):

KPI	Valor Calculado	Fórmula / Fonte	Status
Total de Pedidos	118.936	COUNT(STOREORDER)	✓ Implementado
Receita Total	R\$ 6.197.092,97	SUM(totalamount)	✓ Implementado
Ticket Médio	R\$ 57,12	Receita / Pedidos	✓ Implementado
Clientes Únicos	48.201	COUNT DISTINCT(customerid)	✓ Implementado
Lojas Ativas	43	status = ativo	✓ Implementado
Total de Campanhas	73.000	COUNT(CAMPAIGN)	✓ Implementado
Taxa de Conversão	30,53%	status=4 / enviadas × 100	✓ Implementado
Taxa de Recorrência	Em cálculo	clientes ≥2 pedidos / total	■ Em andamento
ROI Simulado	Em cálculo	(Receita – Custo) / Custo	■ Em andamento
RFM Score	Em cálculo	R + F + M por cliente	■ Em andamento
CAC Simulado	Previsto	Custo campanhas / novos cli.	■ Previsto
LTV Simulado	Previsto	Ticket × Freq. × Retenção	■ Previsto

6. Resultados, Divergências e Ajustes

6.1 Resultados Obtidos vs Esperados

Aspecto	Esperado	Obtido / Atual	Ação Corretiva
Dados demográficos (gênero, nascimento)	Compleitude $\geq 80\%$	Gênero: 37% nulo Nascimento: 94,5% nulo	Preenchimento com 'NI'; filtros de segmentação ajustados para lidar com ausência
Localização das lojas	Campos cidade/estado preenchidos	93,5% nulos em cidade e estado	Análise geográfica de lojas suspensa; foco em análise por cidade do cliente
Valores extremos de pedidos	Pedidos até R\$ 200	Outliers até R\$ 10.000	Filtro de outliers em visualizações; mantidos na base para análise corporativa isolada
Status de pedidos	Mapeamento claro	38 valores distintos sem legenda	Mapeamento em progresso com equipe Cannoli; status críticos identificados
Tempo de resposta do dashboard	< 2 segundos	3-5s sem índices (testes iniciais)	Adição de índices em campos de filtro; cache em memória para consultas frequentes
Relacionamento CAMPAIGN x ORDER	Chave direta campaignid	Join por storeid + customerid + período	Tabela ponte criada em ETL; join validado por consistência de dados

7. Observações Reflexivas do Grupo

Reflexão 1 — KPIs definem a arquitetura

Definir os KPIs cedo (RF-11 a RF-15) foi essencial para montar o Diagrama de Classes e os contratos da API sem precisar refatorar depois. Quando sabemos o que precisamos exibir, fica muito mais fácil decidir quais entidades e campos criar.

Reflexão 2 — Qualidade dos dados foi uma surpresa

O alto índice de campos nulos em dados demográficos (94,5% em data de nascimento, 37% em gênero) foi algo que não esperávamos. A saída foi manter os registros substituindo os nulos por 'NI', já que esses clientes ainda têm valor para os KPIs financeiros.

Reflexão 3 — Segurança no backend, não só no frontend

Percebemos que colocar o controle de perfil só no frontend seria um problema sério: qualquer um poderia chamar os endpoints diretamente. Por isso implementamos o middleware JWT na API antes de avançar nas telas. Essa decisão aparece claramente no Diagrama de Casos de Uso.

Reflexão 4 — Polling em vez de WebSocket no MVP

O RF-07 pede atualização em tempo real. Decidimos começar com polling agendado e deixar o WebSocket para depois, quando o MVP estiver validado. Reduz a complexidade sem comprometer a funcionalidade essencial.

Reflexão 5 — O join entre CAMPAIGN e ORDER não era direto

Não existe uma chave direta entre as tabelas CAMPAIGN e STOREORDER. Resolvemos isso criando uma tabela ponte no ETL, usando storeid + customerid + janela de tempo. Testamos a consistência nos dados reais e a taxa de conversão calculada (30,53%) faz sentido com o volume de campanhas.

Reflexão 6 — Módulo de sugestão de campanha é o ponto mais importante

O RF-18 (sugestão de campanha baseada em RFM) é o que mais diferencia o produto. Desde a primeira entrega já coletamos todos os campos necessários para isso. A implementação está em andamento no backend e vai ser validada contra os 73.000 registros de campanhas da base real.